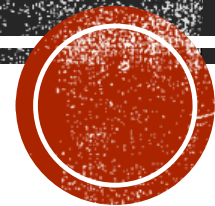




RÉSUMÉ COURS BDA

Essaye de ressemblé les cours de prof




Abdallahi



Données de données : Collection de données cohérents et structurées.

Les avantages de l'organisation en BD :

- Uniformisation de la saisie
- Standardisation des traitements
- Contrôle de la validité de données
- Partage de données entre plusieurs traitements

- **System de gestion de bases de donnees** : Logiciel(s) assurant structuration, stockage, maintenance, mise à jour et consultation des données d'une BD .

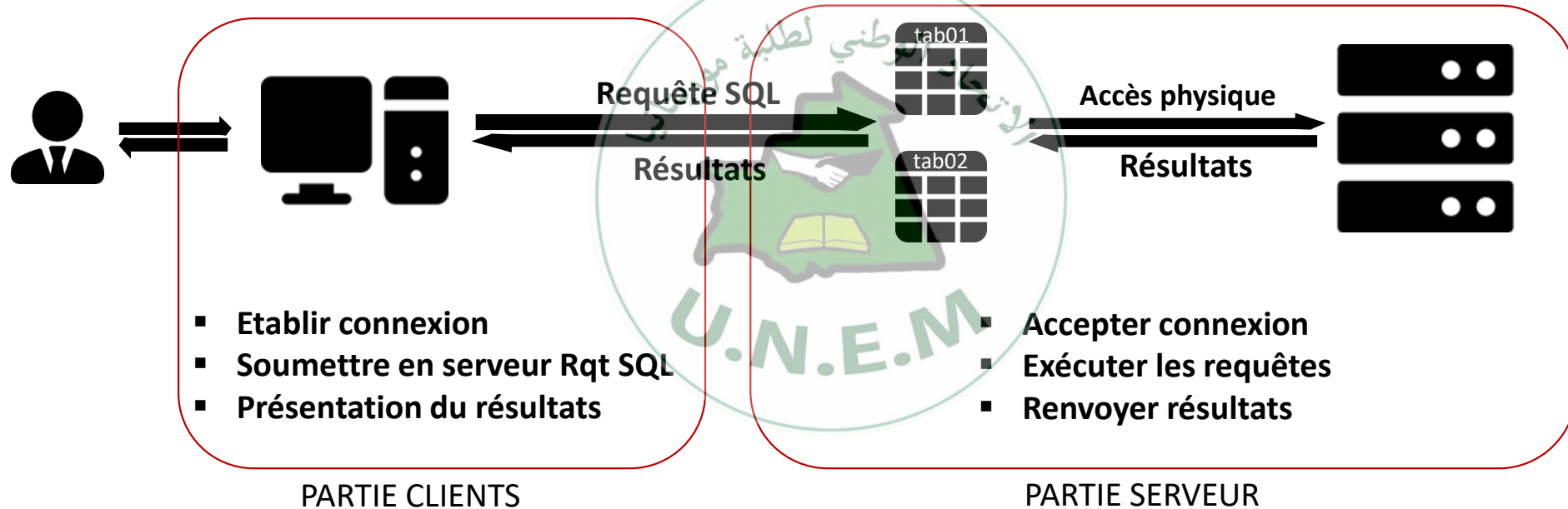
Exemples : Oracle , DB2 , PostgreSQL, MySQL , MariaDB...





Schéma : correspond a un espace logique contenant les objets d'un utilisateur

Schéma illustratif



Principes



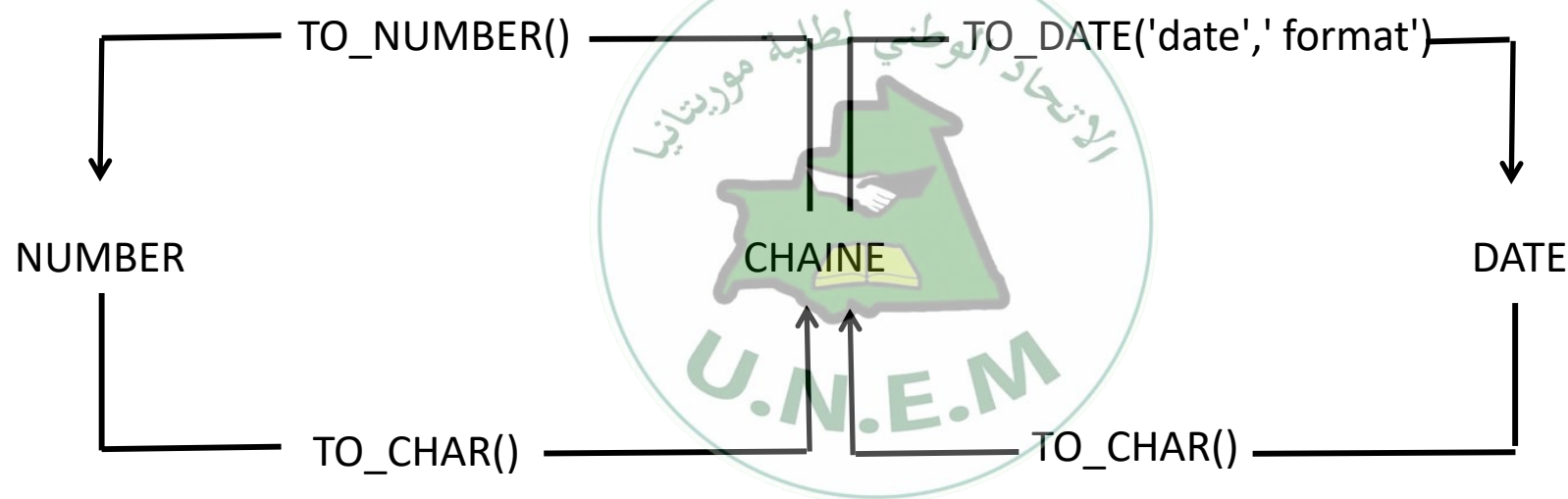
L*PLUS : est un ligne de commande permettant de connecter au bases de données oracle.

- **Transaction** : est un série des opérations lie entre eux qui ne peuvent exécuter qu'ensemble.
- **AUTOCOMMIT** : est un paramètre qui définit si la transaction sont automatique valide ou non.
- **AUTOCOMMIT** : ON => validation automatique.
: OFF => validation manuelle.
- **COMMIT** : est opération qui valide la transaction.
- **ROLLBACK** : annuler la validation et retourner arrière.
- **Jeu de caractère** : définir l'ensemble de symbole reconnue par le système et leur manière d'en codage





fonction de conversion des types des données :



- Format de la date : ('dd/mm/yyyy')
 - _dd : day
 - _mm : month
 - _yyyy : year





établier la connexion avec un base de donnée oracle on exécute la commande :

SQL> CONN[ECT] SYSTEM/PassWord@NameService

- ✓ CONN ou CONNECT : commande de connexion
- ✓ SYS ou SYSTEM : l'utilisateur si vous n'avez pas encore crée d'utilisateur
- ✓ PassWord : le mot de passe par défaut que vous avez rentre dans SYSTEM
- ✓ NameService : nom de service contient :
 - L'adresse hôte de la serveur (@IP)
 - Port
 - Service
 - Protocole c'est 99% de TCP (protocole utilise au couche transport)
 - Nom de base de données

NBs :

- ❖ Si le serveur est local, il n'est pas nécessaire de mentionner le nom du service
- ❖ vous n'avez pas besoin de connaitre tous les composants de NameService, il suffit de connaitre l'alias , puis de mettre dans le fichier «**tnsnames.org**» , ce fichier de configuration du client est utilise pour faciliter et simplifier les connexions des clients Oracle a des services oracle distant.





PL/SQL (Procedural Language)

Un Langage procédural spécifiquement Conçu Pour accepter les Requête SQL au Sein de sa syntaxe

Syntaxe :

DECLARE

/* section de déclaration*/

BEGIN

/*section de traitement*/

EXCEPTION

/*traitement d'exception*/

END;

/

SET SERVEROUTPUT ON : pour active mode
d'affichage les messages de DBMS_OUTPUT

Exemple :

SET SERVEROUTPUT ON;

DECLARE

/*initialization :*/

2 text VARCHAR(30) := 'Hello World';

/*declaration simple :*/

3 text2 VARCHAR(20);

4 BEGIN

5 DBMS_OUTPUT.PUT_LINE(text);

7 END;

8 /

Output : Hello World





PL/SQL OUTPUT

Le package vous permet d'envoyer des messages à partir de procédures stockées et de packages.
Le package est particulièrement utile pour afficher les informations de débogage PL/SQL

Types d'affectation

- Affectation directe :
→ nom_variable type (les types sont type printf oracle (NUMBER(), VARCHAR(n), INT...))
- Affectation par référence
→ nom_variable nom_table.nom_colonne %TYPE

Gestion des exceptions:

EXCEPTION

```
WHEN type d'exception THEN  
    /* */  
WHEN type d'exception THEN  
    /**/  
WHEN OTHER THEN
```





Exemple:

le schema citizen(id , name , age).

NB pour la concatenation on utilise le deux bar: ||

DECLARE

citizen_id citizen.id%type;

citizen_name citizen.name%type;

citizen_age citizen.age%type := 9;

BEGIN

SELECT id, name INTO citizen_id, citizen_name FROM citizen WHERE age = citizen_age;

DBMS_OUTPUT.PUT_LINE ('Citizen id is: ' || citizen_id);

DBMS_OUTPUT.PUT_LINE ('Citizen name is: ' || citizen_name);

EXCEPTION

WHEN no_data_found THEN

dbms_output.put_line ('si aucun enregistrement est retourné par select');

WHEN too_many_rows THEN

dbms_output.put_line ('Cette exception est déclenchée si une requête SELECT récupère plusieurs lignes.');

WHEN others THEN

dbms_output.put_line ('Errors');

END;

/





SELECT INTO :

SELECT

liste_colone

INTO

liste_variable

FROM

nom_table;

Exple:

DECLARE

Champ_1 nom_table.colone%TYPE;

Champ_2 nom_table.colone%TYPE;

BEGIN

SELECT colone1,colone2 INTO champ1,champ2 FROM nom_table;

DBMS_OUTPUT . PUT_LINE (Champ_1);

EXCEPTION

WHEN no_data_found THEN DBMS_OUTPUT.PUT_LINE ('aucun enregistrement');

WHEN too_many_rows THEN DBMS_OUTPUT.PUT_LINE ('trop d'enregistrements');

WHEN others THEN DBMS_OUTPUT.PUT_LINE ('Errors');

END;

/

SELECT INTO pose a problème car la colonne peut avoir plusieurs et la variable champ_1 est mon_valeur dans ce cas il faut gérée d'exception





Declaration d'un type:

DECLARE

```
TYPE nom_type IS RECORD(  
  Champ_1 TYPE;  
  Champ_2 TYPE;  
  .  
  .  
  Champ_n TYPE );  
var nom_type;
```

BEGIN

```
var.Champ_1 := '1234' ;  
var.Champ_2 := 'Ali' ;  
DBMS_OUTPUT . PUT_LINE ( ' var.Champ_1 = ' || var.Champ_1 || ' and var.Champ_2 = ' || var.Champ_2 ) ;
```

END;

/

D'autre maniere :

DECLARE

```
ligne nom_table.%ROWTYPE;
```

BEGIN

```
SELECT INTO ligne FROM nom_table;  
DBMS_OUT.OUTLINE(ligne.nom_colone);
```

END;

/





Structures repetitives

```
Loop  
  /*Traitement*/  
Exit Condition;  
End Loop
```

Exple

SQL > DECLARE

1. Compteur Number(1):=0;
2. BEGIN
3. Loop
4. DBMS_output.put_Line('Compteur');
5. i:=i+1;
6. Exit i=10;
7. END;
8. /

PL/SQL

Structures alternatives ou conditionnelles

```
If condition then  
  /*traitement */  
ELSEIF condition then  
  /*traitement */  
ELSE  
  /*traitement */  
END IF;
```

Exemple :

SQL> DECLARE

1. a number :=5 ;
2. b number:=4;
3. BEGIN
4. if a=b THEN
5. DBMS_output.put_line(a || 'ET' || b || ' sont égaux);
6. ELSE
7. DBMS_output.put_line(a || 'ET' || b || 'sont différents');
8. ENDIF
9. END; /





Structures répétitives

, For et While

condition LOOP

/*traitement*/

END LOOP;

WHILE condition LOOP

/*traitement*/

end loop;

Exple

SQL> DECLARE

2 i NUMBER:=0;

3 BEGIN

4 LOOP

5 DBMS_output.put_line(i);

6 i:=i+1;

7 exit when i>10;

8 END LOOP;

9 for i in 1..10 loop

10 DBMS_output.put_line(i);

11 END LOOP;

12 while i<=10 loop

13 DBMS_output.put_line(i);

14 END LOOP;

15 END ;

16 /

PL/SQL

Structures alternatives ou conditionnelles

- Pour demander l'entrée de l'utilisateur nous utilisons : & avant la variable.
- Pour désactiver mode de vérification nous utilisons : SET VERIFY OFF sinon (SET VERIFY ON) va afficher l'ancien et nouveau valeur par défaut ;

```
SQL> DECLARE
1 i NUMBER(1):=5;
2 BEGIN
3 CASE &i
4 WHEN 1 THEN DBMS_OUTPUT.PUT_LINE('Lundi');
5 WHEN 2 THEN DBMS_OUTPUT.PUT_LINE('Mardi');
6 WHEN 3 THEN DBMS_OUTPUT.PUT_LINE('Mercredi');
7 WHEN 4 THEN DBMS_OUTPUT.PUT_LINE('Jeudi');
8 WHEN 5 THEN DBMS_OUTPUT.PUT_LINE('Vendredi');
9 WHEN 6 THEN DBMS_OUTPUT.PUT_LINE('Samdi');
10 WHEN 7 THEN DBMS_OUTPUT.PUT_LINE('Dmanche');
11 ELSE
12 DBMS_OUTPUT.PUT_LINE('What??');
13 END CASE;
14 END;
15 /
Entrez une valeur pour i : 98
ancien 4 : CASE &i
nouveau 4 : CASE 98
What??

Procédure PL/SQL terminée avec succès.
```



on :

```
CREATE OR REPLACE FUNCTION nom_fonction ( [parametres ] )  
RETURN /*type*/ IS  
    /* declaration des variables locales */  
BEGIN  
    /* instructions */  
END;
```

Exemple:

```
SQL> CREATE OR REPLACE FUNCTION sommeEntiers(a NUMBER,b NUMBER)  
2 RETURN NUMBER IS  
3  
4 BEGIN  
5 RETURN a+b;  
6 END ;  
7 /
```

Fonction créée.

```
SQL> select sommeEntiers(6,7) FROM dual;
```

```
SOMMEENTIERS(6,7)
```

```
-----  
13
```

```
SQL>
```



CURSOR:

Le curseur oracle sert d'espace memoire permettant d'enregistrer le resultat d'une requette de selection(SELECT INTO) quel soit sa taille

Syntaxe:

DECLARE

CURSOR nom_cursor IS

SELECT liste_colone FROM nom_table ;

ligne nom_cursor%rowtype ;

BEGIN

OPEN nom_cursor ;

LOOP

FETCH nom_cursor INTO ligne ;

EXIT WHEN nom_cursor%NOTFOUND;

DBMS_OUTPUT.PUT_LINE (ligne.nom_colone) ;

END LOOP ;

CLOSE nom_cursor ;

END;

/

- %NOTFOUND renvoi → TRUE si les ligne termine

Exemple:

```
1 DECLARE
2 CURSOR nom_curs IS
3 SELECT * FROM vols2 ;
4 ligne nom_curs%ROWTYPE;
5 BEGIN
6 OPEN nom_curs;
7 LOOP
8 FETCH nom_curs INTO ligne;
9 EXIT WHEN nom_curs%NOTFOUND;
10 DBMS_OUTPUT.PUT_LINE(ligne.no_vol);
11 END LOOP;
12 CLOSE nom_curs;
13* END;
it100
it101
it102
it103
it104
```